# Finite-Difference Jacobian for the Bistatic Model

How the code approximates the Jacobian of the bistatic measurement model by central finite differences, why the error behaves as it does, and how the step sizes were chosen.

## What it computes

Column *i* of the Jacobian is the partial derivative of all three outputs (excess range, range rate, azimuth) with respect to input *x_i*:

`J[:, i] = ∂h/∂x_i`

The finite-difference version approximates each partial derivative by nudging one input at a time and watching how the outputs move. There are 18 inputs and two nudges each, so 36 model evaluations per Jacobian.

## Derivation

Take the model h as a function of one input, with the others fixed, and Taylor-expand around x:

- `h(x + δ) = h + δ h′ + (δ²/2) h″ + (δ³/6) h‴ + …`
- `h(x − δ) = h − δ h′ + (δ²/2) h″ − (δ³/6) h‴ + …`

### Forward difference

Rearranging the first line gives

`(h(x+δ) − h(x)) / δ = h′ + (δ/2) h″ + …`

The error is proportional to δ, so it is first-order accurate.

### Central difference

Subtract the second line from the first. The even-order terms (h and h″) cancel:

`(h(x+δ) − h(x−δ)) / (2δ) = h′ + (δ²/6) h‴ + …`

The error is proportional to $\delta^2$, so it is second-order accurate. This is what the code uses, at the cost of one extra evaluation per input (36 instead of 18, or 19 if the nominal evaluation is shared).

## Step size: truncation vs. roundoff

Two errors compete as δ changes:

- **Truncation error** grows with δ ($\delta^2 h'''/6$ for central).
- **Roundoff error** grows as δ shrinks. Each model evaluation carries a rounding error of about $\varepsilon \cdot |h|$, and dividing the difference by 2δ amplifies it to about $\varepsilon \cdot |h|/\delta$.

The total is roughly `δ²|h‴|/6 + ε|h|/δ`, which is minimized near `δ* ~ (ε|h|/|h‴|)^(1/3)`. For central differences this is typically about $10^{-5}$ times the problem's length scale, versus about $10^{-8}$ times it for forward differences.

In this model the length scale is the range ($10^4$–$10^5$ m), so $h''' \sim h'/R^2$, which makes the relative truncation error about $\delta^2/(6R^2)$. The ECEF coordinates are about $4\times10^6$ m, so each coordinate is only known to about $10^{-9}$ m in double precision. That sets the roundoff floor.

### Step-size sweep

I ran a sweep on the target x-position column, comparing against the analytic Jacobian:

| Step δ (m) | Forward rel. error | Central rel. error |
|---|---|---|
| $10^{-6}$ | ~$10^{-5}$ | ~$10^{-5}$ |
| $10^{-4}$ | ~$10^{-6}$ | ~$10^{-6}$ |
| $10^{-2}$ | ~$10^{-7}$ | ~$10^{-9}$ |
| **1** | $10^{-5}$ | **~$10^{-9}$** |
| $10^{2}$ | $10^{-3}$ | ~$10^{-5}$ |
| $10^{4}$ | 0.1–0.5 | ~0.07 |

At tiny steps both methods are limited by roundoff, with errors around $10^{-5}$ to $10^{-6}$. As δ grows, forward error rises linearly and central error quadratically, which is the textbook behaviour. Central at 0.01–1 m is the sweet spot, where it matches the analytic Jacobian to about $10^{-9}$. The code's choice of 1 m for positions sits comfortably in that range.

# Details in the code

## Step sizes

`step = h_pos if (i // 3) % 2 == 0 else h_vel` picks 1 m for the position blocks and 0.01 m/s for the velocity blocks. The state is ordered pos, vel, pos, vel, pos, vel, so block index `i // 3` is even for positions. The model is exactly linear in velocity (range rate) or independent of it (range, azimuth), so the velocity step size only affects roundoff, not truncation.

## Azimuth wrap-around

The azimuth is wrapped to $[-\pi, \pi)$, so a tiny perturbation near $\pm\pi$ could jump by $2\pi$ and produce a huge bogus slope. The fix is to wrap each perturbed azimuth relative to the nominal one:

```
hp[2] = h0[2] + _wrap_pi(hp[2] - h0[2])
```

The perturbed value then stays on the same branch as the nominal value.

## Frame dependence

Because the whole model is re-evaluated, the finite-difference Jacobian includes the ENU frame rotation (the "frame term" in the analytic derivation) automatically.

# Alternatives

- **Analytic Jacobian:** exact and about 18× cheaper, but must be updated by hand if the model changes.
- **Automatic differentiation** (JAX or similar): exact and automatic, but it would require porting the model off pymap3d.
- **Complex-step differentiation:** attractive in principle, but it doesn't work through `atan2` or `pymap3d` here.